Індивідуальне завдання

1. Тема індивідуального завдання

Розробка сучасного адаптивного та доступного вебзастосунку з використанням HTML, CSS, JavaScript/TypeScript та REST API.

2. Мета

Розробити, протестувати та опублікувати власний вебзастосунок, продемонструвавши практичне володіння основними технологіями дисципліни: HTML5, CSS, JavaScript, TypeScript, REST API та сучасними інструментами frontend-розробки.

3. Основні вимоги

Виконати такі вимоги:

- самостійно обрати та погодити предметну область;
- створити семантичну HTML5-структуру;
- реалізувати сучасне CSS-оформлення та responsive design;
- забезпечити базову Web Accessibility відповідно до WCAG 2.2;
- реалізувати інтерактивність за допомогою JavaScript;
- інтегрувати щонайменше один REST API;
- реалізувати loading та error states;
- використати TypeScript для типізації частини застосунку;
- застосувати модульну структуру проєкту та Vite/NPM;
- виконати debugging, testing та code review;
- опублікувати готовий вебзастосунок online;
- надати Git-репозиторій, README та матеріали до захисту.

Формат вебзастосунку

Реалізувати вебзастосунок із простою структурою у форматі:

- landing page з інтерактивними елементами та даними з API; або
- простого MPA з 2–3 сторінок, наприклад: головна → каталог → детальна інформація.

Не вимагається реалізація:

- авторизації та реєстрації користувачів;
- власного backend;
- бази даних;
- складної системи адміністрування;
- повноцінного CRUD;
- складної бізнес-логіки.

Основна увага приділяється frontend-розробці, роботі з API, якості коду, accessibility та responsive design.

4. Приклади предметних областей та тем вебзастосунків

Обрати одну із запропонованих тем або запропонувати власну, погодивши її з викладачем.

№	Предметна область / тема	Публічне API	Орієнтовний формат
1	**Каталог книжок** — пошук та перегляд інформації про книги	Open Library API — відкритий API з даними про книги, авторів і видання, з пошуком та обкладинками	MPA: каталог → детальна інформація
2	**Каталог серіалів** — пошук серіалів та перегляд інформації про них	TVmaze API — REST API для пошуку шоу, епізодів, акторського складу та рейтингової інформації	MPA: пошук → список → details
3	**Каталог рецептів** — пошук, фільтрація та перегляд рецептів	DummyJSON Recipes API — готовий набір даних із рецептами, пошуком, фільтрацією, сортуванням і пагінацією	Landing + каталог або MPA
4	**Сервіс прогнозу погоди для подорожей** — прогноз для обраного міста	Open-Meteo API — weather forecast та geocoding API без API key	Landing: пошук міста → прогноз
5	**Каталог порід собак** — перегляд порід та галерея фотографій	Dog API — відкритий API з переліком порід, пошуком за породою та зображеннями	Landing + галерея / MPA
6	**Каталог Pokémon** — список персонажів та детальна інформація	PokéAPI — безкоштовний відкритий REST API з даними про персонажів, типи, навички та спрайти	MPA: каталог → details
7	**Віртуальна галерея мистецтва** — пошук та перегляд творів мистецтва	Art Institute of Chicago API — публічний REST-style API у JSON з даними про твори мистецтва	Landing + gallery + details

№	Предметна область / тема	Публічне API	Орієнтовний формат
8	**Каталог Нобелівських лауреатів** — пошук та перегляд інформації про лауреатів і премії	Nobel Prize API — публічний API з даними про лауреатів, премії та відповідні об'єкти	MPA: список → details
9	**Конвертер валют** — порівняння та конвертація валют	Frankfurter API — API курсів валют з поточними та історичними значеннями	Landing page
10	**Каталог космічних запусків та подій** — перегляд майбутніх запусків	Launch Library 2 API — публічний API з інформацією про запуски, події, місії та космічні апарати	Landing + список запусків + details

Рекомендації щодо вибору теми

Обрати тему, що дає можливість продемонструвати:

- отримання даних через REST API;
- пошук, фільтрацію або сортування;
- відображення списку даних;
- перегляд детальної інформації;
- щонайменше 3 інтерактивні сценарії;
- loading та error states;
- responsive та accessible interface.

5. Етапи виконання та розподіл часу

№	Етап	Години
1	Аналіз предметної області, структура проєкту, семантичний HTML5	5
2	CSS, Box Model, cascade, specificity, CSS Variables	5
3	SEO, Web Accessibility, responsive design	8
4	JavaScript, DOM, Events, модулі, debugging	9
5	REST API, Fetch/async-await, loading/error states	9
6	TypeScript та типізація API/коду	8
7	Типізація існуючого коду, рефакторинг та code review	6
8	Vite/NPM, Git, ESLint, Prettier, testing, deployment	6
9	Комплексне доопрацювання та підготовка до захисту	6
	Разом	**62**

6. Результати, які необхідно подати

Обов'язково:

1. Git-репозиторій із вихідним кодом.
2. Опублікований вебзастосунок із доступним URL.
3. README, що містить призначення проєкту, використані технології, API, інструкцію запуску, опис основної функціональності та короткий технічний звіт із описом реалізованих рішень.
4. Захист роботи з демонстрацією застосунку та поясненням коду.

7. Мінімальні вимоги до вебзастосунку

- містити семантичну HTML5-структуру;
- мати адаптивний інтерфейс;
- використовувати CSS Variables та сучасні CSS Layout;
- містити щонайменше 3 інтерактивні сценарії;
- отримувати та відображати дані з REST API;
- обробляти loading та error states;
- містити TypeScript-код;
- мати базові тести;
- пройти базову перевірку accessibility;
- бути опублікованим online.

8. Захист

Під час захисту:

- продемонструвати роботу вебзастосунку;
- показати Git-репозиторій;
- пояснити структуру проєкту;
- пояснити ключові фрагменти HTML, CSS, JavaScript/TypeScript;
- продемонструвати роботу REST API;
- продемонструвати debugging та testing;
- обґрунтувати основні технічні рішення;
- відповісти на запитання щодо власного коду.

9. Оцінювання

Критерій	Бали
HTML, семантика та структура	10
CSS, layout та responsive design	15
SEO та Web Accessibility	10
JavaScript, DOM та Events	15
REST API та асинхронний JS	15
TypeScript	10
Testing, debugging та code quality	20
Deployment, README та оформлення проєкту	5
Разом	**100**

> **Обов'язкова умова зарахування.** Для зарахування роботи надати працездатний проєкт, Git-репозиторій, опубліковану версію вебзастосунку та продемонструвати здатність пояснити власний код і використані технічні рішення.

10. Подання роботи

Роботу необхідно надіслати електронним листом на адресу denys.matsevych@oa.edu.ua.

Тема листа:

```
[Веб] Індивідуальне завдання
```

У листі вказати:

- посилання на Git-репозиторій;
- посилання на опублікований вебзастосунок.